Use custom instructions in VS Code

Custom instructions provide reusable context that helps AI follow your coding practices and project requirements. Instead of repeating guidelines in every chat prompt, store them in Markdown files that you can share with your team or reuse across projects.

This article helps you choose an instruction type and location, configure when instructions apply, and verify that your agent uses them. For a guided setup workflow, see Configure AI for your codebase.

Instruction support depends on the agent harness selected for your session. Each harness uses its own discovery rules and file formats. If you select the Local harness, VS Code uses the Local harness-specific instruction settings described in this article.

> **Tip**
>
> Use the Agent Customizations editor to discover, create, and manage customizations for the selected agent harness. Run **Chat: Open Customizations** from the Command Palette.

> **Note**
>
> Custom instructions are not taken into account for inline suggestions as you type in the editor.

Types of instruction files

Select the intended harness before you open the Agent Customizations editor. The editor shows the customizations available to that harness.

Choose a format

Use an instruction format supported by the selected harness.

Expand table

Harness	Recommended project instructions	Targeted instructions
Copilot	`.github/copilot-instructions.md` or `AGENTS.md`	`.github/instructions/**/*.instructions.md`
Anthropic Claude	`CLAUDE.md`	Markdown files in `.claude/rules`
OpenAI Codex	`AGENTS.md`	`AGENTS.md` files in subfolders

> **Note**
>
> **For Local harness sessions:** Project instructions can use `.github/copilot-instructions.md`, `AGENTS.md`, or `CLAUDE.md`. Targeted instructions can use `.github/instructions/**/*.instructions.md` or Markdown files in `.claude/rules`. The Local harness-specific settings in the sections below control discovery.

`AGENTS.md` is not specific to Codex. It is a cross-agent format that you can use as the shared project instructions file when your selected harnesses support it. Some harnesses also recognize additional formats.

If your team uses multiple harnesses, use a shared format that they all support when possible. When you need separate files, keep shared requirements consistent and avoid contradictory copies.

Choose a scope

Store instructions at the narrowest scope that matches how you want to use and share them.

Expand table

Scope	Use it for	Storage
Workspace	Project conventions shared with contributors	A supported folder in the repository or session working folder
User	Personal preferences across projects	A user folder supported by the selected harness
Organization	Centrally managed requirements for supported Copilot sessions	GitHub organization settings

User instructions in harness folders, such as `~/.copilot/instructions` and `~/.claude/rules`, do not roam through Settings Sync. Local harness instructions stored in your VS Code profile can roam through Settings Sync.

Choose activation behavior

Project instructions are automatically included according to the selected harness. Use targeted instructions when guidance applies only to specific files or tasks:

- **File pattern**: VS Code automatically attaches an `.instructions.md` file when its `applyTo` pattern matches a file that the agent creates or modifies.
- **Task relevance**: a descriptive `description` helps the agent decide whether to load an instructions file for the current task.

- **Manual**: attach an instructions file to an individual chat request.

For Claude rules, use the `paths` frontmatter property instead of `applyTo`.

To reference files or URLs in instructions, use Markdown links. Relative file paths resolve from the instructions file. Use `/` path separators to keep instructions portable across operating systems.

Use a .github/copilot-instructions.md file

For Copilot sessions, use `.github/copilot-instructions.md` for project-wide guidance. Store the file in the `.github` folder at the repository root.

The Local harness also discovers this workspace file when `github.copilot.chat.codeGeneration.useInstructionFiles` is enabled.

Use `copilot-instructions.md` for:

- Coding style and naming conventions that apply across the project.
- Technology stack declarations and preferred libraries.
- Architectural patterns to follow or avoid.
- Security requirements and error handling approaches.
- Documentation standards.

For personal, always-on instructions in Copilot sessions, use `~/.copilot/copilot-instructions.md`.

Follow these steps to create a `.github/copilot-instructions.md` file in your workspace:

1. Create a `.github/copilot-instructions.md` file at the root of your repository. If needed, create a `.github` directory first.
2. Add concise, project-specific instructions in Markdown.

▶ **Example: General coding guidelines**

Use .instructions.md files

You can create file-based instructions with `*.instructions.md` Markdown files. VS Code automatically attaches files with an `applyTo` pattern that matches the files being changed. The agent can also load an instructions file on demand when its `description` matches the current task.

Use `.instructions.md` files for:

- Different conventions for frontend and backend code.
- Language-specific guidelines in a monorepo.
- Framework-specific patterns for specific modules.
- Specialized rules for test files or documentation.

Instructions file locations

The supported location depends on the session type and selected harness.

Expand table

Session and scope	Default file location
Copilot workspace	`.github/instructions`
Anthropic Claude workspace	`.claude/rules`
Copilot user	`~/.copilot/instructions`
Anthropic Claude user	`~/.claude/rules`

Use the Agent Customizations editor to create user instructions in a location supported by the selected harness. To move profile-based instructions to a supported harness user folder, use user customization migration.

> **Note**
>
> **For Local harness sessions:** Workspace instruction files use `.github/instructions` or `.claude/rules`, and user instructions use VS Code profile storage.
>
> Prefer native harness folders over custom search locations. The `chat.instructionsFilesLocations` setting remains supported and is not deprecated, and the editor forwards instructions from additional configured locations to Agent Host harnesses. Custom locations are not recommended for Copilot workflows because GitHub Copilot CLI and the GitHub Copilot app do not inherit editor settings. Move existing instructions to native harness locations to use them across Copilot clients.

VS Code searches these folders recursively, which enables you to organize instructions files in subdirectories. For example, you can group instructions by team, language, or module:

```
.github/instructions/
  frontend/
    react.instructions.md
    accessibility.instructions.md
  backend/
    api-design.instructions.md
  testing/
    unit-tests.instructions.md
```

> **Tip**
>
> In a monorepo, enable `chat.useCustomizationsInParentRepositories` to discover instructions from the parent repository root. Learn more about parent repository discovery.

Instructions file format

For `.instructions.md` files, YAML frontmatter controls how the instructions are discovered and applied:

Expand table

Field	Required	Description
`name`	No	Display name shown in the UI. Defaults to the file name.
`description`	No	Describes the tasks for which the file is relevant. Include it for on-demand discovery.
`applyTo`	No	Glob pattern that automatically applies the instructions to matching files, relative to the workspace root. Use `**` to match all files.

The body contains the instructions in Markdown format.

```
---
name: 'Python testing'
description: 'Use when creating or updating Python unit tests.'
applyTo: '**/*.py'
---
# Python testing

* Use `pytest` fixtures from `tests/conftest.py` instead of creating duplicate set
* Name tests `test_<behavior>_<condition>`.
* Run `python -m pytest <test-file>` after changing a test.
```

If you omit both `description` and `applyTo`, attach the file manually when you want to use it. For Claude rules, use the `paths` property instead of `applyTo`.

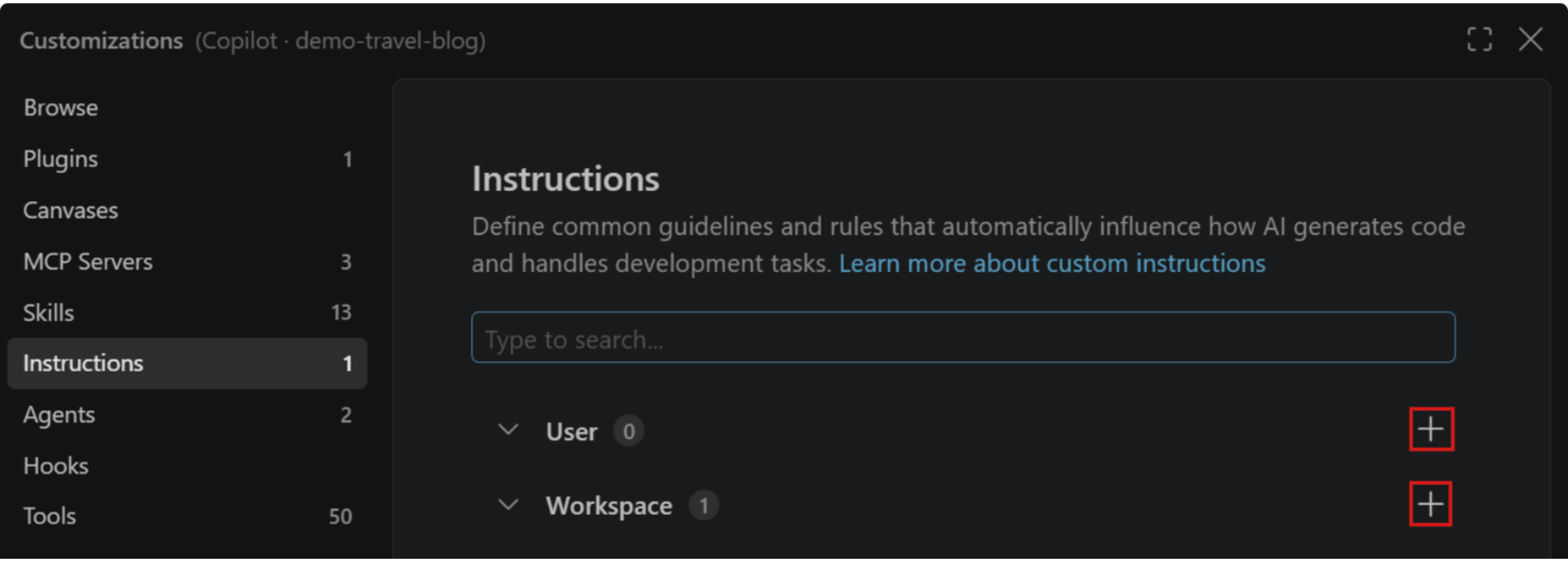
Create an instructions file

When you create an instructions file, choose whether to store it at workspace or user scope. Workspace instructions apply only to that workspace, while user instructions are available across multiple workspaces for the corresponding harness.

To create an instructions file:

1. Select the agent harness that should use the instructions.
2. In the Chat view, select **Open Customizations** in the title bar, or run **Chat: Open Customizations** from the Command Palette (⇧⌘P).
3. In the Agent Customizations editor, select **Instructions**.
4. Select **New Instruction** (plus icon) next to the **Workspace** or **User** group, depending on where you want to store the file.
5. Select a supported location and enter a file name.
6. Add the frontmatter and Markdown instructions, and then save the file.

You can also run **Chat: New Instructions File** from the Command Palette (⇧⌘P).





Generate an instructions file with AI

In chat, describe the convention or guideline you want to enforce and ask the agent to create an instructions file. This works for every harness, including **Copilot**. Confirm that the generated file uses the format and location for the selected harness.

With **Local** selected, you can also use these shortcuts, which aren't available for other harnesses:

- Type `/create-instructions` in chat and describe the convention or guideline.
- Type `/init` to generate project-wide instructions. In the Chat view, the **Customize Your Agent** box on the **Overview** page of the Agent Customizations editor also uses `/init`.

Review generated instructions before you use them. Verify commands, paths, and conventions against the repository.

Use an AGENTS.md file

`AGENTS.md` is a cross-agent format for project guidance. Multiple harnesses, including Copilot and OpenAI Codex, support it. The Local harness also supports this format. Place the primary file at the repository root.

Use `AGENTS.md` when:

- You work with multiple compatible AI coding agents and want to share one set of instructions.
- You want subfolder-level instructions that apply to specific parts of a monorepo.

For Copilot sessions, instructions are discovered at the repository root, the session's working directory, and intermediate directories between them. Copilot combines applicable instructions from `AGENTS.md` and other supported instruction files. Avoid conflicting guidance because there is no general precedence order between these files. For more information, see Copilot instruction discovery.

> **Note**
>
> **For Local harness sessions:** The `chat.useAgentsMdFile` setting controls whether instructions from workspace-root `AGENTS.md` files are automatically attached to chat requests and is enabled by default. This setting does not control instruction discovery in other harnesses.

Use multiple AGENTS.md files

Use nested `AGENTS.md` files when different folders need different guidance. As Copilot works with files in subfolders, it discovers `AGENTS.md` files along their directory paths. Other harnesses use their own discovery and activation rules.

> **Note**
>
> **For Local harness sessions:** Set both `chat.useAgentsMdFile` and `chat.useNestedAgentsMdFiles` to `true` to discover nested `AGENTS.md` files. The nested-file setting is disabled by default. When both settings are enabled, VS Code lists these files with their folder locations so the agent can load relevant instructions for the task. Neither setting controls Copilot instruction discovery.

> **Tip**
>
> For folder-specific instructions, you can also use multiple `.instructions.md` files with different `applyTo` patterns that match the folder structure.

Use a CLAUDE.md file

For Anthropic Claude sessions, use `CLAUDE.md` at the repository root for project-wide guidance. The Anthropic Claude harness also supports additional native locations and formats, such as `.claude/CLAUDE.md` and `.claude/rules`.

The Local harness searches for Claude instructions in these locations when `chat.useClaudeMdFile` is enabled:

Expand table

Location	Description
Workspace root	`CLAUDE.md` in the root of your workspace
`.claude` folder	`.claude/CLAUDE.md` in your workspace
User home	`~/.claude/CLAUDE.md` for personal instructions across all projects
Local variant	`CLAUDE.local.md` for local-only instructions (not committed to version control)

To enable or disable support for `CLAUDE.md` files, configure the `chat.useClaudeMdFile` setting.

> **Note**
>
> For `.claude/rules` instructions files, use a `paths` property instead of `applyTo` for glob patterns, following the Claude rules format. The `paths` property accepts an array of glob patterns and defaults to `**` when omitted.

Generate custom instructions for your workspace

The Agent Customizations editor can start a chat that analyzes your repository and creates instructions for the selected harness. Review the result before saving it because generated commands, architecture details, or conventions might be incomplete.

For a complete generation and verification workflow, see Configure AI for your codebase.

Verify your instructions

Test instructions with a representative task where your project guidance affects the result.

1. Select the intended harness, open the Agent Customizations editor, and confirm that the instructions file is listed. This verifies discovery, but not whether the instructions are followed.
2. Start a new chat with the same harness and repository.
3. Ask the agent to complete a small task with a clear success criterion.
4. Expand **References** in the response and confirm that the expected instructions were used.
5. Review the result and tool activity to check that the agent followed the relevant conventions and validation steps.

If the file is missing or the result does not follow the instructions, see Why is my instructions file not being applied?.

Other instruction sources

Share custom instructions across teams

To share custom instructions across multiple workspaces and repositories within your GitHub organization, you can define them at the GitHub organization level.

Supported Copilot sessions automatically include organization instructions that your account can access. Organization instructions are additive to user and repository instructions.

To enable discovery of organization-level custom instructions, set `github.copilot.chat.organizationInstructions.enabled` to `true`.

Learn how you can add custom instructions for your organization in the GitHub documentation.

Sync Local harness instructions across devices

VS Code can sync Local harness instructions stored in your user profile by using Settings Sync. User instructions in harness folders, such as `~/.copilot/instructions` and `~/.claude/rules`, do not roam through Settings Sync.

To sync profile instructions, enable Settings Sync and run **Settings Sync: Configure** from the Command Palette (`⇧⌘P`). Select **Prompts and Instructions** from the list of settings to sync.

Specify instructions for generated content

> **Note**
>
> Settings-based code generation and test generation instructions are deprecated as of VS Code 1.102. Use file-based instructions instead.

For code review, commit messages, and pull request descriptions, you can still use VS Code settings to define custom instructions. These settings accept an array of objects with either a `text` property (inline instruction) or a `file` property (path to a Markdown file).

Expand table

Scenario	Setting
Review selected code	`github.copilot.chat.reviewSelection.instructions`
Commit messages	`github.copilot.chat.commitMessageGeneration.instructions`
Pull request descriptions	`github.copilot.chat.pullRequestDescriptionGeneration.instructions`

Resolve conflicting instructions

Applicable instruction sources are additive. Do not depend on a file order or precedence rule to resolve conflicts because discovery and merge behavior can differ by harness.

Keep shared requirements consistent across user, repository, and organization instructions. Remove duplicate guidance and resolve contradictions at their source.

For example, suppose organization instructions require an approved HTTP client, repository instructions identify the project's wrapper for that client, and personal instructions request a different library. A targeted instruction for API files might add another conflicting rule. Don't add a new instruction or assume that one source overrides the others.

1. Start a new chat with the intended harness and ask for a small change that matches the targeted instruction.
2. Use the Agent Customizations editor and Agent Debug Logs to check which instruction sources are discovered and whether they contain errors.
3. Expand **References** in the response to confirm which instructions were included for the request.
4. Compare the generated code, tool activity, and validation results with the repository requirement.
5. Resolve the contradiction in the source files. Remove or narrow the personal rule, and align the repository and targeted instructions with the reviewed organization requirement.
6. Repeat the same task in another new chat and compare the observed result.

For more diagnostic steps, see Troubleshoot AI in VS Code and inspect the instructions sent in the model request.

Tips for writing effective instructions

- Keep your instructions short and self-contained. Each instruction should be a single, simple statement. If you need to provide multiple pieces of information, use multiple instructions.
- Include the reasoning behind rules. When instructions explain *why* a convention exists, the AI makes better decisions in edge cases. For example: "Use `date-fns` instead of `moment.js` because moment.js is deprecated and increases bundle size."
- Show preferred and avoided patterns with concrete code examples. The AI responds more effectively to examples than to abstract rules.
- Focus on non-obvious rules. Skip conventions that standard linters or formatters already enforce.
- For task or language-specific instructions, use multiple `*.instructions.md` files per topic and apply them selectively by using the `applyTo` property.
- Store project-specific instructions in your workspace to share them with other team members and include them in your version control.
- Reuse and reference instructions files in your prompt files and custom agents to keep them clean and focused, and to avoid duplicating instructions.

Frequently asked questions

Why is my instructions file not being applied?

> **Tip**
>
> Use the Agent Customizations editor to confirm that the selected harness discovers the file. To inspect loading details and errors, run **Developer: Open Agent Debug Logs** from the Command Palette, or select **Show Agent Debug Logs** from the ellipsis (...) menu in the Chat view. Learn more about troubleshooting agent customizations.

If your instructions file is not being applied, check the following:

- Verify that the intended agent harness is selected.
- Verify that your instructions file is in a supported location for that harness and scope.
- For `*.instructions.md` files, check that `applyTo` matches a file the agent creates or modifies, or that `description` clearly identifies the relevant task. If neither applies, attach the file manually.
- For the Local harness, check that relevant settings are enabled. These include `chat.includeApplyingInstructions` for pattern-based instructions, `chat.includeReferencedInstructions` for linked instructions, and `chat.useAgentsMdFile` for `AGENTS.md`.
- Expand **References** in the chat response to check which instructions were used.

For advanced diagnostics, inspect the instructions sent in the model request.

How do I find an instruction file?

1. Select the agent harness that should use the instruction.
2. Run **Chat: Open Customizations** from the Command Palette (`⇧⌘P`).
3. Select **Instructions**, and then select the instruction to view its source and location.

If the instruction is missing, run **Developer: Open Agent Debug Logs** to inspect discovery and loading errors.